

Final Project – Todo API Documentation

PHP REST API

Authentication APIs

1. Sign up: POST

- **url:** /apis/v1/auth/sign_up.php
- **Body (Form Data - all):**
 - first_name (text)
 - last_name (text)
 - username (text)
 - password (text)
 - image (file)
- **Response (JSON):**
 - result
 - error (if exists)
- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - first_name: minimum 3 characters and maximum 20 characters
 - last_name: minimum 3 characters and maximum 20 characters
 - username: minimum 3 characters and maximum 20 characters
 - password: minimum 6 characters and maximum 30 characters
 - username cannot be existed before
 - image file is required

2. Sign in: POST

- **url:** /apis/v1/auth/sign_in.php
- **Body (JSON - all):**
 - username (text)
 - password (text)

Final Project – Todo API Documentation

PHP REST API

- **Response (JSON):**
 - result
 - error (if exists)
 - user
 - ✓ id
 - ✓ username
 - ✓ token
 - ✓ profile_url
- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - username: minimum 3 characters and maximum 20 characters
 - password: minimum 6 characters and maximum 30 characters

3. Update password: PUT

- **url:** /apis/v1/auth/update_password.php
- **Headers:**
 - Authorization: Bearer token
- **Body (JSON - all):**
 - old_password
 - new_password
- **Response (JSON):**
 - result
 - error (if exists)
 - user
 - ✓ new_token

Final Project – Todo API Documentation

PHP REST API

- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - When getting the id from token, make sure to check that user is existed in **users** table.
 - old_password: minimum 6 characters and maximum 30 characters
 - new_password: minimum 6 characters and maximum 30 characters
 - old_password is different than new_password

4. Update profile: PUT

- **url:** /apis/v1/auth/update_profile.php
- **Headers:**
 - Authorization: Bearer token
- **Body (Form Data – one or more):**
 - username (optional - text)
 - image (optional - file)
- **Response (JSON):**
 - result
 - error (if exists)
 - user
 - ✓ new_token
 - ✓ new_profile_url (if profile requested to be changed)
 - ✓ new_username (if username requested to be changed)
- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - When getting the id from token, make sure to check that user is existed in **users** table.

Final Project – Todo API Documentation

PHP REST API

- username: minimum 3 characters and maximum 20 characters (if requested to be changed)
- Username or image (or both) are required in the form data.

5. Delete account: DELETE

- **url:** /apis/v1/auth/delete_account.php
- **Headers:**
 - Authorization: Bearer token
- **Response (JSON):**
 - result
 - error (if exists)
- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - When getting the id from token, make sure to check that user is existed in **users** table.

Todo APIs

1. Add Todo: POST

- **url:** /apis/v1/todo/add.php
- **Headers:**
 - Authorization: Bearer token
- **Body (Form Data – all):**
 - title (text)
 - description (text)
 - image (file)

Final Project – Todo API Documentation

PHP REST API

- **Response (JSON):**
 - result
 - error (if exists)
 - todo
 - ✓ title
 - ✓ description
 - ✓ is_done
 - ✓ image_url
- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - When getting the id from token, make sure to check that user is existed in **users** table.
 - title: minimum 2 characters and maximum 100 characters
 - description: minimum 2 characters and maximum 255 characters

2. Get all todos: GET

- **url:** /apis/v1/todo/ or /apis/v1/todo/index.php
- **Headers:**
 - Authorization: Bearer token
- **Response (JSON):**
 - result
 - error (if exists)
 - todos (array of json of all todos for this user)
- **Response Code: Appropriate HTTP status code**

Final Project – Todo API Documentation

PHP REST API

- **Validation/notes:**

- When getting the id from token, make sure to check that user is existed in **users** table.
- Return all todos for this specific user, not for any other user.

3. Delete todo: DELETE

- **url:** /apis/v1/todo/delete_todo?todo_id=id

- **Headers:**

- Authorization: Bearer token

- **Response (JSON):**

- result
- error (if exists)

- **Response Code: Appropriate HTTP status code**

- **Validation/notes:**

- When getting the id from token, make sure to check that user is existed in **users** table.
- Check if todo needed to be deleted is existed in **todos** table.
- User can delete a todo only if they own it.

4. Update todo: PUT

- **url:** /apis/v1/todo/update.php

- **Headers:**

- Authorization: Bearer token

- **Response (Form Data - one or more):**

- id (int - required)
- title (optional - text)
- description (optional - text)

Final Project – Todo API Documentation

PHP REST API

- image (optional - file)
- **Response Code: Appropriate HTTP status code**
- **Validation/notes:**
 - When getting the id from token, make sure to check that user is existed in **users** table.
 - Check if todo needed to be updated is existed in **todos** table.
 - User can update a todo only if they own it
 - Title, description or image (or all of them) are required in the form data.